

■ Cours Express — Catégorie 4

■ Practical Pythonic & Async System Design (Mistral FDE - PRIORITÉ #3)

1. ■ Standards d'API REST & Robustesse HTTP

A. Codes de Statut HTTP Inappropriés (FastAPI)

Problème : Renvoyer HTTP 200 OK avec un corps {'error': 'Failed'} en cas d'échec.

Impact Observabilité : Fausse le monitoring (100% de succès affiché sur les métriques alors que le service plante).

■ Code Buggué	■ Correction Propre
<pre>@app.post('/predict') def predict(data: dict): if not data.get('text'): return {'error': 'Missing text'} # ■ HTTP 200 renvoyé !</pre>	<pre>@app.post('/predict') def predict(data: PredictSchema): if not data.text: raise HTTPException(status_code=400, detail='Missing text') # ■ Proper HTTP 400</pre>

B. Retries d'Écritures sans Clé d'Idempotence

Problème : Re-tenter aveuglément des requêtes HTTP POST/PUT en cas d'erreur 5xx.

Impact Scalabilité : Entraîne des tempêtes de retries (Thundering Herd), des doublons de facturation et la corruption d'état.

■ Code Buggué	■ Correction Propre
<pre>def create_chat_session(user_id: str): # ■ Retry sans idempotence = risques de doublons return requests.post('/sessions', json={'user_id': user_id})</pre>	<pre>def create_chat_session(user_id: str, idempotency_key: str): headers = {'Idempotency-Key': idempotency_key} return httpx.post('/sessions', headers=headers, json={'user_id': user_id})</pre>

2. ■ Performance Async & Traitements de Fond

A. Requêtes N+1 d'API ou de BDD dans des Boucles

Problème : Interroger une BDD ou API à chaque itération d'une boucle for.

Impact Scalabilité : Latence linéaire en O(N) allers-retours réseau. Écroule les pools de connexion.

■ Code Buggué	■ Correction Propre
<pre>for doc_id in doc_ids: doc = db.query(Document).get(doc_id) # ■ N+1 queries BDD !</pre>	<pre># ■ Requête groupée en un seul aller-retour docs = db.query(Document).filter(Document.id.in_(doc_ids)).all()</pre>

B. Tâches Longues Bloquant le Thread API (Fast vs Slow Paths)

Problème : Traiter de gros OCR ou de la génération longue dans le cycle synchrone HTTP.

Impact Scalabilité : Saturation des workers Web et crashes par 504 Gateway Timeout.

■ Code Buggué	■ Correction Propre
<pre>@app.post('/ocr') def process_ocr(file: UploadFile): text = heavy_pdf_ocr(file) # ■ Bloque le worker 30 secondes ! return {'text': text}</pre>	<pre>@app.post('/ocr', status_code=202) def process_ocr(file: UploadFile, bg_tasks: BackgroundTasks): job_id = uuid4() bg_tasks.add_task(heavy_pdf_ocr_job, job_id, file) # ■ Fast Path 202 Accepted return {'job_id': job_id}</pre>

■ Réflexes Oraux pour l'Entretien Mistral (Jour J)

1. 'Lever `HTTPException` avec les bons codes' → Ne jamais renvoyer HTTP 200 avec du JSON d'erreur.
2. 'Contrôler l'Idempotence sur les Retries' → Utiliser des `Idempotency-Key` (UUID) sur les requêtes POST/PUT.
3. 'Éliminer les requêtes N+1' → Utiliser des requêtes de batching (`IN (...)`) au lieu de boucler sur la BDD.
4. 'Séparer Fast et Slow Paths' → Renvoyer HTTP 202 Accepted pour les tâches lourdes et déléguer en Background Job.